

Denotational Semantics for Effectful Gradually Typed Languages using Guarded Type Theory

ERIC GIOVANNINI, University of Michigan, USA

MAX S. NEW, University of Michigan, USA

Gradually-typed languages allow for soundly mixing static and dynamically typed programming styles and support migration from a dynamic to static typing style. Establishing the desired metatheoretic properties for real-world gradually-typed languages has proved challenging. In recent work, Giovannini et al. gave a denotational semantics for a simple gradually-typed lambda calculus using the tools of guarded type theory, and furthermore showed that their semantics was sufficient for establishing the crucial metatheoretic property known as *graduality*, which states that migrating a program by making the types more precise does not fundamentally change the meaning of the program except to possibly catch more type errors.

In this work, we address two limitations of the work of Giovannini et al. First, we generalize their model to allow for languages with algebraic effects, such as I/O, global state, nondeterminism, etc. Second, we relate the denotational semantics to a more familiar operational semantics.

ACM Reference Format:

Eric Giovannini and Max S. New. 2025. Denotational Semantics for Effectful Gradually Typed Languages using Guarded Type Theory. 1, 1 (September 2025), 8 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Gradually typed languages [5, 7] allow for safe interoperability between static and dynamic typing disciplines in the same codebase, and they support the smooth, *gradual* migration from dynamic typing to static typing via the addition of explicit typing information. One of the fundamental theorems for gradually typed languages is *graduality*, also known as the *dynamic gradual guarantee*, originally defined by Siek, Vitousek, Cimini, and Boyland [4, 6]. Informally, graduality says that migrating code from dynamic to static typing should only allow for the introduction of static or dynamic type errors, and not otherwise change the behavior of the program.

Establishing graduality for a given cast calculus has traditionally been a tedious endeavor. One common approach involves defining a step-indexed logical relation over an operational semantics [?]; these methods are inherently dependent on the particular calculus being modeled and offer little opportunity for reuse of mathematical abstractions across different developments. Another method, developed by New and Licata [?], constructs a denotational semantics using *classical* domain theory. While compositional, this approach suffers from a lack of expressiveness: classical domain-theoretic techniques appear incapable of modeling viciously self-recursive structures such as higher-order store [1], which are common in real-world gradually-typed languages. Thus, it remained an open question whether one could give a semantics for gradual typing that combined

Authors' Contact Information: Eric Giovannini, Electrical Engineering and Computer Science, University of Michigan, USA, ericgio@umich.edu; Max S. New, Electrical Engineering and Computer Science, University of Michigan, USA, maxsnew@umich.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2025/9-ART

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

the expressiveness of the step-indexed approach with the compositional nature of denotational semantics.

Recently, Giovannini et al. [3] answered this question affirmatively by constructing a denotational semantics for a simple gradually-typed programming language using the tools of guarded type theory [?], and proved that their model validates the graduality property. Their model combines the expressive power of step-indexing techniques with the compositional nature of denotational semantics. However, the semantics developed by Giovannini et al. suffers from the limitation that it is a single model for a quite bare-bones gradually-typed language. Real-world languages can include effects, polymorphism, dependent types, etc. Furthermore, it is unclear how to relate their model to a more typical operational semantics for a cast calculus. It is desirable to establish an adequacy result relating their denotational semantics to an operational model. More specifically, writing $\llbracket M \rrbracket$ for the denotation of a term M , adequacy states that an equality of denotations $\llbracket M \rrbracket = \llbracket N \rrbracket$ implies that M and N behave the same operationally. This allows one to use facts proved in the denotational semantics to reason about the operational behavior of terms.

In this work, we seek to address the limitations of the prior model by extending their denotational semantics to account for arbitrary algebraic effects, and establishing an adequacy theorem relating the resulting denotational model to an operational semantics.

The level of generality needed to model arbitrary algebraic effects means that the semantics we produce is correspondingly generic. This is true both denotationally and operationally. On the denotational side, our guarded semantics models effectful computations as a kind of *inductive-guarded interaction tree*. These are a generalization of guarded interaction trees [2] that allow for inductive occurrences of the variable in addition to guarded-recursive ones. The (guarded) graduality theorem we obtain is essentially a statement relating term precision to the behavior of the interaction trees that the terms denote. Likewise, we construct a generic effectful operational semantics following the methodology of Kavvos [?]. While this approach is applicable to any signature of effects, what is missing is a way to turn the semantics into a more traditional “runnable” operational semantics. For example, a big-step semantics for a language with global state would normally involve a transition relation between pairs of a term and state.

The missing step in both the operational and denotational semantics is supplied by a suitable notion of algebra that interprets the guarded interaction trees as a concrete element of an “observation” type.

More concretely, the organization of this paper is as follows:

- (1) Given a signature of algebraic effects, we define a cast calculus supporting those effects. We give an inequational theory in the form of axioms for type and term precision, and define an operational semantics for the cast calculus, drawing inspiration from recent work of Kavvos [?].
- (2) We show how to construct a denotational semantics for such a cast calculus by adapting the construction of Giovannini et al. [3] to account for arbitrary algebraic effects.
- (3) Following the technique of the aforementioned work by Kavvos, we prove an adequacy result relating the operational and denotational models in the expected manner.

1.1 Metatheoretic Preliminaries

2 A Generic Syntax for Gradual Typing with Effects

To begin, we fix a framework for specifying gradual cast calculi with algebraic effects. We use a syntax based on fine-grain call-by-value presented in Figure 1.

$$\begin{array}{lcl}
\text{Types } A & ::= & \text{Nat} \mid ? \mid A \multimap A' \mid A \times A' \\
\text{Type Precision } c & ::= & r(A) \mid cc' \mid \text{Inj}_{\multimap} \mid \text{Inj}_{\text{N}} \mid \text{Inj}_{\times} \mid c \multimap c' \mid c \times c' \\
\text{Values } V & ::= & x \mid \text{up } c \, V \mid \text{zro} \mid \text{suc } V \mid \lambda x. M \mid (V, V') \\
\text{Terms } M, N & ::= & \mathcal{U} \mid \text{up } c \, M \mid \text{dn } c \, M \mid \text{zro} \mid \text{suc } M \mid \lambda x. M \\
& & \mid M N \mid (M, N) \mid \text{let } (x, y) = M \text{ in } N \mid \text{op}(V; x. M) \\
\text{Contexts } \Gamma & ::= & \cdot \mid \Gamma, x : A \\
\text{Ctx Precision } \Delta & ::= & \cdot \mid \Delta, x : c
\end{array}$$

$$\frac{\Gamma \vdash M : A \quad c : A \sqsubseteq A'}{\Gamma \vdash \text{up } c \, M : A'} \text{Up} \qquad \frac{\Gamma \vdash N : A' \quad c : A \sqsubseteq A'}{\Gamma \vdash \text{dn } c \, N : A} \text{DN}$$

$$\frac{\text{op} : ! \sigma ? \tau \quad \Gamma \vdash V : \sigma \quad \Gamma, x : \tau \vdash M : A}{\Gamma \vdash \text{op}(V; x. M) : A} \text{EFF} \qquad \Gamma \vdash \mathcal{U} : A$$

$$r(A) : A \sqsubseteq A \qquad \frac{c : A \sqsubseteq A' \quad c' : A' \sqsubseteq A''}{cc' : A \sqsubseteq A''} \qquad \text{Inj}_{\multimap} : ? \multimap ? \sqsubseteq ? \qquad \text{Inj}_{\text{N}} : \text{Nat} \sqsubseteq ?$$

$$\text{Inj}_{\times} : ? \times ? \sqsubseteq ? \qquad \frac{c_i : A_i \sqsubseteq A'_i \quad c_o : A_o \sqsubseteq A'_o}{c_i \multimap c_o : (A_i \multimap A_o) \sqsubseteq (A'_i \multimap A'_o)} \qquad \frac{c_1 : A_1 \sqsubseteq A'_1 \quad c_2 : A_2 \sqsubseteq A'_2}{c_1 \times c_2 : (A_1 \times A_2) \sqsubseteq (A'_1 \times A'_2)}$$

$$r(A)c \equiv c \qquad c \equiv cr(A') \qquad c(c'c'') \equiv (cc')c'' \qquad r(A_i \multimap A_o) \equiv r(A_i) \multimap r(A_o)$$

$$r(A_1 \times A_2) \equiv r(A_1) \times r(A_2) \qquad (c_i \multimap c_o)(c'_i \multimap c'_o) \equiv (c_i c'_i \multimap c_o c'_o)$$

$$(c_1 \times c_2)(c'_1 \times c'_2) \equiv (c_1 c'_1 \times c_2 c'_2)$$

Fig. 1. Syntax for Gradual Typing with Effects

3 Operational Semantics

In this section, we define a small-step operational semantics for the generic cast calculus introduced in Section 2. Our approach is based on the recent work of Kavvos [?], who gave a generic operational semantics for a language with algebraic effects.

Following the work of Kavvos, our operational semantics has two kinds of reductions. There are *pure reductions*, which comprise the usual β reductions in an ordinary lambda calculus as well as reductions involving casts. There are also *impure reductions*, which result from the program making a request to its environment and receiving a value in response.

The pure reductions are labelled by a natural number $i \in \{0, 1\}$, which intuitively tracks the number of recursive unfoldings involved in each step. The source of such steps is the downcast from the dynamic type to a function, since in the denotational semantics the function is only available later and hence the downcast involves an observable computational step. We track the recursive unfolding of terms in the operational semantics to help us relate the operational and denotational semantics, which we address in a subsequent section.

The impure reductions do not involve any recursive unfolding, so they are not labeled by a natural number. A general impure reduction has the form

$$M \xrightarrow{\text{op} : ! V ? W} N,$$

which means that M sends value V to the environment and continues as N upon receiving response W .

There are β rules for functions, pairs, and bind, which are standard. Next are the rules involving casts. We first have rules involving upcasts and downcasts of functions and products. When a function V_f is upcasted, it doesn't immediately step. When the upcasted function is applied to a value, the value is downcasted according to the domain and applied to the original function V_f , and then the result is upcasted according to the codomain. The dual holds for a function that is downcasted. When a pair of values (V_1, V_2) is upcasted, we simply upcast each component of the pair, and likewise for downcasting. Next we have rules for up/down casts involving composed precision derivations, i.e., $\text{Inj}_\perp(c)$ and $\text{Inj}_\times(c)$. These rules decompose such casts into the cast for c and the primitive injection casts $\text{Inj}_\rightarrow$ or $\text{Inj}_\times$. For example, to upcast a term using the precision derivation $\text{Inj}_\perp(c)$ where $c : A \sqsubseteq (? \rightarrow ?)$, we first upcast the term by c , and then upcast the result by $\text{Inj}_\rightarrow : ? \rightarrow ? \sqsubseteq ?$. Finally, we have the rules involving downcasting a value that was cast to $?$ using the injection casts $\text{Inj}_\rightarrow$, $\text{Inj}_\perp(c)$, and $\text{Inj}_\times(c)$. In each case, we check that the tag of the upcast matches the tag of the downcast, e.g., the upcast is $\text{Inj}_\rightarrow$ and the downcast is $\text{Inj}_\rightarrow$. If the tags match, then the cast steps to the original value; otherwise, the result is an error. Note that when downcasting a dynamically-typed function, we record one recursive unfolding. In the denotational semantics, this corresponds to inserting a θ .

Next, we have our first impure reduction rule. This rule states that for an operation op with request type σ and response type τ , then for any possible request value V and response value W , the term $\text{op}(V; x.M)$ steps to $M[W/x]$. This represents the fact that the environment has responded with value W , after which the program continues as M with W in place of x . The final two rules are simply congruence rules allowing us to perform pure and impure reductions under an arbitrary evaluation context.

3.1 A Guarded Interaction Tree Semantics

4 Denotational Semantics

In this section, we discuss how to adapt the model of Giovannini et al. [3] to accommodate algebraic effects. Recall that in their model, pure values were modeled as elements of a *predomain*, and effectful computations were modeled as elements of an *error domain*, which is a predomain equipped with algebraic operations supporting error and computational steps. Our approach will be to generalize the notion of error domain to be an algebra of a suitable endofunctor corresponding to the signature of effects being modeled. Then the free error domain construction will be generalized to construct the free such algebraic structure, as an *inductive-guarded interaction tree*.

In addition to generalizing error domains, we need to make an important technical modification to the model of Giovannini et al. This change is motivated by our goal of relating the denotational and operational semantics via an adequacy result. In op. cit., the authors mention that the denotations of casts in their model involves additional perturbations (i.e., computational steps) in order to place certain terms involving casts in lock-step. These additional steps ultimately did not matter for the final result, since the graduality property is step-insensitive. However, intensionally, these casts behave differently from the corresponding casts without the extra steps. This presents a difficulty when relating their denotational model to an operational semantics: in order for the two semantics to be in lock-step, we must define the operational semantics in such a way that the casts take additional steps just as they do in the denotational semantics. But this requirement is not desirable, as it restricts the kinds of operational semantics to which our theorem applies to those that happen to feature this strange cast semantics. Users of our framework would need to contort their operational semantics into this form if they would like to apply the adequacy result.

Beta rules:

$$\frac{}{(\lambda x : A.M)(V) \rightarrow^0 M[V/x]} \text{BETA} \quad \frac{}{\text{split}((V_1, V_2); x_1, x_2.M) \rightarrow^0 M[V_1, V_2/x_1, x_2]} \text{SPLIT}$$

$$\frac{}{\text{ret } V \text{ to } x.M \rightarrow^0 M[V/x]} \text{BIND}$$

Casts (functorial actions and composition):

$$\frac{c_i : A_i \sqsubseteq A'_i \quad c_o : A_o \sqsubseteq A'_o \quad \Gamma \vdash V_f : A_i \rightarrow A_o \quad \Gamma \vdash V : A'_i}{((\text{up } (c_i \rightarrow c_o)V_f) V) \rightarrow^0 (\text{up } c_o (V_f (\text{dn } c_i V)))} \text{UP-ARR}$$

$$\frac{c_1 : A_1 \sqsubseteq A'_1 \quad c_2 : A_2 \sqsubseteq A'_2}{\text{up } (c_1 \times c_2)(V_1, V_2) \rightarrow^0 (\text{up } c_1 V_1, \text{up } c_2 V_2)} \text{UP-PAIR}$$

$$\frac{c_1 : A_1 \sqsubseteq A'_1 \quad c_2 : A_2 \sqsubseteq A'_2}{\text{dn } (c_1 \times c_2)(V_1, V_2) \rightarrow^0 (\text{dn } c_1 V_1, \text{dn } c_2 V_2)} \text{DN-PAIR}$$

$$\frac{c : A \sqsubseteq (? \rightarrow ?) \quad \Gamma \vdash M : A}{(\text{up } (\text{Inj}_{\rightarrow}(c))M) \rightarrow^0 (\text{up } \text{Inj}_{\rightarrow} (\text{up } c M))} \text{INJ}_{\rightarrow} \text{COMPUP}$$

$$\frac{c : A \sqsubseteq (? \rightarrow ?) \quad \Gamma \vdash M : ?}{(\text{dn } (\text{Inj}_{\rightarrow}(c))M) \rightarrow^0 (\text{dn } c (\text{dn } \text{Inj}_{\rightarrow} M))} \text{INJ}_{\rightarrow} \text{COMP DN}$$

$$\frac{c : A \sqsubseteq (? \times ?) \quad \Gamma \vdash M : A}{(\text{up } (\text{Inj}_{\times}(c))M) \rightarrow^0 (\text{up } \text{Inj}_{\times} (\text{up } c M))} \text{INJ}_{\times} \text{COMPUP}$$

$$\frac{c : A \sqsubseteq (? \times ?) \quad \Gamma \vdash M : ?}{(\text{dn } (\text{Inj}_{\times}(c))M) \rightarrow^0 (\text{dn } c (\text{dn } \text{Inj}_{\times} M))} \text{INJ}_{\times} \text{COMP DN}$$

Casts involving dyn:

$$\frac{}{\text{dn } \text{Inj}_{\mathbb{N}} (\text{up } \text{Inj}_{\mathbb{N}} V) \rightarrow^0 V} \quad \frac{c : A \sqsubseteq (? \times ?)}{\text{dn } \text{Inj}_{\times} (c) (\text{up } \text{Inj}_{\times} (c) V) \rightarrow^0 V}$$

$$\frac{c : A \sqsubseteq (? \rightarrow ?)}{\text{dn } \text{Inj}_{\rightarrow} (c) (\text{up } \text{Inj}_{\rightarrow} (c) V) \rightarrow^1 V} \quad \frac{\text{tag}_1, \text{tag}_2 \in \{\mathbb{N}, \times, \rightarrow\}}{\text{dn } \text{Inj}_{\text{tag}_1} (\text{up } \text{Inj}_{\text{tag}_2} V) \rightarrow^0 \mathcal{U}}$$

Effects and congruence rules:

$$\frac{\text{op} : !\sigma ? \tau}{\text{op}(V; x.M) \xrightarrow{\text{op} : !V ? W} M[W/x]} \text{EFF} \quad \frac{i \in \{0, 1\} \quad M \rightarrow^i N}{E[M] \rightarrow^i E[N]} \text{CONG}$$

$$\frac{M \xrightarrow{\text{op} : !V ? W} N}{E[M] \xrightarrow{\text{op} : !V ? W} E[N]} \text{EFFCONG}$$

Fig. 2. Operational Semantics (Selected Rules)

Our remedy for this situation is to note that there is an additional degree of freedom in the model of Giovannini et al. that up to this point has remained unexploited. Leveraging this, we define two *different* cast semantics, one where the casts involve perturbations, and one where they do not. The former is used in constructing the semantic proofs of the cast axioms, where being in lock-step is crucial. The latter is what we use for the denotational term semantics of casts, allowing the denotations of casts to be in lock-step with the casts in a typical operational model, i.e., where the casts do not take additional steps. While the intensional behavior of these two kinds of casts are distinct, they are extensionally related in that they are *weakly bisimilar*, which is sufficient for the final proof of graduality.

4.1 Generalizing Error Domains

In this section, we discuss how to build a model of gradual typing from scratch given a collection of effects presented algebraically. Our goal will be to generalize the notion of *error domain* from prior work to accommodate these effects. We begin by reviewing the key ingredients in the denotational semantics of Giovannini et al. [3]. For further discussion, see their paper.

Definition 4.1. A **predomain** A consists of a set A along with two relations:

- A partial order $\sqsubseteq_A$.
- A reflexive, symmetric “bisimilarity” relation $\approx_A$.

Definition 4.2. An **error domain** B consists of a predomain B along with the following data:

- A distinguished “error” element $\mathcal{U}_B \in B$ such that $\mathcal{U} \sqsubseteq_B x$ for all x
- A morphism of predomains $\theta_B: \triangleright B \rightarrow B$
- For all $x, y: B$ with $x \approx_B y$, we have $\theta_B(\text{next } x) \approx_B y$

A **morphism of predomains** $f: A \rightarrow A'$ is a function between the underlying sets that is both *monotone* (if $x \sqsubseteq_A x'$, then $f(x) \sqsubseteq_{A'} f(x')$), and *bisimilarity-preserving* (if $x \approx_A x'$, then $f(x) \approx_{A'} f(x')$).

A **morphism of error domains** is a morphism of the underlying predomains that preserves the error element and the θ map. For an error domain B , we define the predomain morphism $\delta_B := \theta_B \circ \text{next}$.

A **predomain relation** $c: A \multimap A'$ is a relation between A and A' that is upward closed in A' and downward closed in A , as defined in the previous section (no condition is required for bisimilarity). The “reflexive” predomain relation is written $r(A)$ and is given by the ordering $\sqsubseteq_A$. The relational composition of two predomain relations is also a predomain relation, and $r(A)$ is the identity for this composition.

Error domain relations are predomain relations that are additionally a congruence with respect to the algebraic operations θ and $\mathcal{U}$:

Definition 4.3. An **error domain relation** $d: B \multimap B'$ is a relation d on the underlying predomains satisfying

- ($\mathcal{U}$ congruence): $\mathcal{U}_B d \mathcal{U}_{B'}$.
- (θ congruence): For all $\tilde{x}$ in $\triangleright B$ and $\tilde{y} \in \triangleright B'$, if $\triangleright_t(\tilde{x} d \tilde{y}_t)$, then $(\theta_B(\tilde{x}) d \theta_{B'}(\tilde{y}))$.

There is again a reflexive error domain relation $r(B)$ given by the ordering. The relational composition of two error domain relations in general is not an error domain relation (θ congruence in particular is not preserved), but we instead use an alternative “free composition” $dd': B \multimap B''$ that is inductively generated from the relational composition.

Lastly, we have the notion of a square $f \sqsubseteq_{\mathcal{U}}^{c_i} g$ between predomain morphisms and of a square between error domain morphisms:

Definition 4.4. Let $f : A_i \rightarrow A_o$ and $g : A'_i \rightarrow A'_o$ be predomain morphisms, and let $c_i : A_i \multimap A'_i$ and $c_o : A_o \multimap A'_o$ be predomain relations. There is a *predomain square* $f \sqsubseteq_{c_o}^{c_i} g$ if for all $x \ c_i \ y$, we have $f(x) \ c_o \ g(y)$.

Likewise, an *error domain square* $\phi \sqsubseteq_{d_o}^{d_i} \psi$ is defined similarly, with no extra interaction with the algebraic structure.

We next review how the model of Giovannini et al. validates the graduality property. The authors of that work took as a starting point a semantics with a notion of representable relations, which in prior work was sufficient for interpreting the four cast rules crucial for graduality. However, as the authors show, the notion of representability cannot be applied unchanged to the guarded setting because of a fundamental tension between transitivity and step-insensitive reasoning. As a result, Giovannini et al. worked with a weakened notion that they called *quasi-representability*, which is essentially representability up to an explicit delay term. They then developed a theory of these delay terms, which they called *syntactic perturbations*.

In the remainder of this section, we discuss the changes we must make to accommodate arbitrary algebraic effects. It turns out that the key question is how to generalize the notion of error domain.

Consider a signature Σ of algebraic effects given by a type Op of operation names, and for each $\text{op} : \text{Op}$ a pair of *predomains* In_{op} and Out_{op} representing respectively the request and response types for op . This signature induces a polynomial endofunctor Σ on the category of predomains and predomain morphisms. Our generalization of error domains will be an algebra B of the endofunctor Σ , equipped with a distinguished error element $\mathcal{U}$ and a distinguished $\theta_B : \triangleright B \rightarrow B$. Since Σ acts on predomains, an algebra of the endofunctor is a predomain B equipped with a morphism of predomains $\Sigma(B) \rightarrow B$. This means in particular that B has ordering and bisimilarity relations. As in the definition of an error domain from prior work, we require that $\mathcal{U}$ is the smallest element in the ordering, that θ respects the relations (i.e., it is a morphism of predomains), and that $\delta_B := \theta_B \circ \text{next}$ is bisimilar to the identity.

More formally, we make the following definition:

Definition 4.5. Let $\Sigma = (\text{Op}, \text{In}, \text{Out})$ be a signature as defined above. A Σ -domain is a predomain B along with, for each $\text{op} \in \text{Op}$, a morphism of predomains

$$\alpha^{\text{op}} : \text{In}_{\text{op}} \times (\text{Out}_{\text{op}} \rightarrow B) \rightarrow B.$$

Furthermore, the predomain B has a distinguished element $\mathcal{U}_B$ such that $\mathcal{U}_B \sqsubseteq_B x$ for all $x \in B$, along with a distinguished morphism of predomains $\theta_B : \triangleright B \rightarrow B$, such that if $x \approx_B y$, then $\delta_B \approx_B y$ where $\delta_B := \theta_B \circ \text{next}$.

As with ordinary error domains, there are corresponding notions of morphisms, relations, and squares of Σ -domains. Before introducing these, we first discuss the action of Σ on predomain morphisms and predomain relations.

Let $c : A \multimap A'$ be a predomain relation. The action of the endofunctor Σ on c , written $\Sigma(c)$ is a predomain relation between $\Sigma(A)$ and $\Sigma(A')$.

Now we can introduce the definitions of morphisms and relations of Σ -domains.

Definition 4.6. Fix Σ as above, and let B and B' be Σ -domains. A *morphism of Σ -domains* from B to B' is a morphism ϕ of the underlying predomains that preserves the algebraic structure. Spelling this out concretely, we have: (1) $\phi(\alpha_B^{\text{op}}(x, k)) = \alpha_{B'}^{\text{op}}(x, \phi \circ k)$, (2) $\phi(\mathcal{U}_B) = \mathcal{U}_{B'}$, and (3) $\phi \circ \theta_B = \theta_{B'} \circ (\triangleright \phi)$.

A *relation of Σ -domains* $d : B \multimap B'$ is a relation $|d|$ of the underlying predomains that is a congruence with respect to the algebraic structure. In particular: (1) If $x \ (\Sigma(|d|)) \ y$, then $\alpha_B^{\text{op}}(x) \ |d| \ \alpha_{B'}^{\text{op}}(y)$. (2) $\mathcal{U}_B \ |d| \ \mathcal{U}_{B'}$, and (3) If $\triangleright_t(\tilde{x}_t \ |d| \ \tilde{y}_t)$, then $\theta_B(\tilde{x}) \ |d| \ \theta_{B'}(\tilde{y})$.

A square of Σ -domains $\phi \sqsubseteq_{d'}^d \phi'$ is defined as a square of the underlying predomains.

Remark 1. The requirement that $\mathcal{U}_B$ be related to $\mathcal{U}_{B'}$ in the definition of relation may seem weaker than the corresponding requirement in the definition of a relation of error domains in the previous section, where we required that $\mathcal{U}_B \mid d \mid y$ for *all* $y \in B'$. However, the former implies the latter using the fact that predomain relations are closed under the ordering on both sides and that $\mathcal{U}_{B'} \sqsubseteq y$ for all y .

5 Relating the Two Semantics

Having specified the operational and denotational semantics, we now seek to prove a theorem relating them. Our goal is to show that the denotational model is *adequate* for the operational semantics, that is, if the denotations of two terms are equal, then their operational behavior is the same.

To this end, we will define a logical relation between (closed) syntactic terms of type A and their denotations as elements of $F[[A]]$, where F is the functor constructing the free error-effect algebra as discussed in Section 4.

6 Discussion

References

- [1] Lars Birkedal, Kristian Støvring, and Jacob Thamsborg. 2009. Realizability Semantics of Parametric Polymorphism, General References, and Recursive Types. In *Foundations of Software Science and Computational Structures, Berlin*. 456–470. doi:10.1007/978-3-642-00596-1_32
- [2] Dan Frumin, Amin Timany, and Lars Birkedal. 2024. Modular Denotational Semantics for Effects with Guarded Interaction Trees. *Proceedings of the ACM on Programming Languages (PACMPL)* 8, POPL, Article 12 (Jan. 2024), 30 pages. doi:10.1145/3632854
- [3] Eric Giovannini, Tingting Ding, and Max S. New. 2025. Denotational Semantics of Gradual Typing using Synthetic Guarded Domain Theory. *Proc. ACM Program. Lang.* 9, POPL, Article 27 (Jan. 2025), 30 pages. doi:10.1145/3704863
- [4] Max S. New and Amal Ahmed. 2018. Graduality from Embedding-Projection Pairs. *Proceedings of the ACM on Programming Languages (PACMPL)* 2, ICFP, Article 73 (jul 2018). doi:10.1145/3236768
- [5] Jeremy G. Siek and Walid Taha. 2006. Gradual Typing for Functional Languages. In *Scheme and Functional Programming Workshop (Scheme)*. 81–92.
- [6] Jeremy G. Siek, Michael M. Vitousek, Matteo Cimini, and John Tang Boyland. 2015. Refined Criteria for Gradual Typing. In *1st Summit on Advances in Programming Languages (SNAPL 2015)*, Vol. 32. 274–293. doi:10.4230/LIPIcs.SNAPL.2015.274
- [7] Sam Tobin-Hochstadt and Matthias Felleisen. 2006. Interlanguage Migration: From Scripts to Programs. In *Dynamic Languages Symposium (DLS)*. 964–974.